CS 202
Advanced Operating Systems

Winter 26

Lecture 4: Synchronization

Instructor: Chengyu Song

Need for Synchronization

- Access to share resources
  - Memory (threads, kernel)
  - OS resources: files, terminal, communication channel, network, etc.
  - Computer systems
  - Physical resources
  - …

- It is important to coordinate their progress to ensure proper outcome

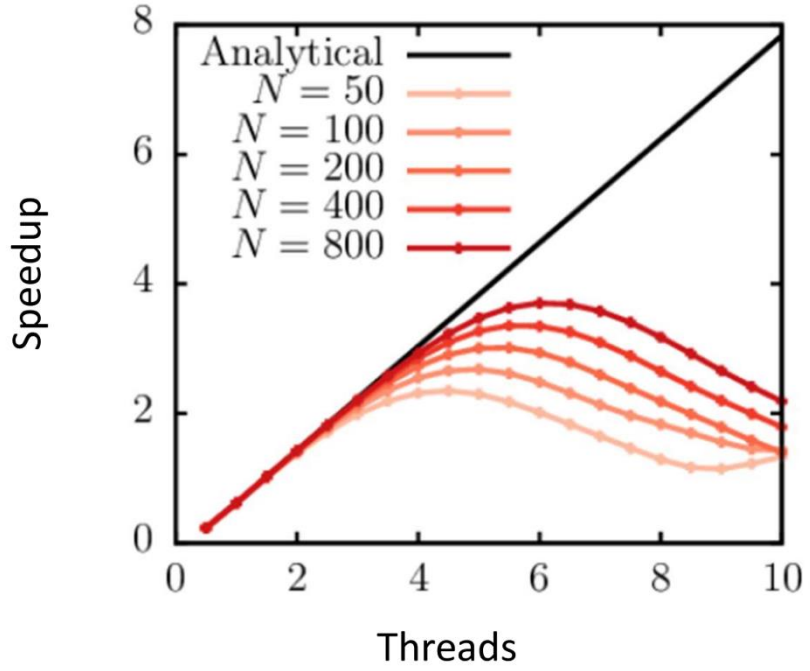
MCS Lock: hardware is not magic

- Pure software lock is hard
  - Hard to get right
  - Hard to prove right
- Even better, software locks don't really work
  - Compiler optimizations and hardware (out-of-order execution) can reorder memory accesses from different threads
- Hardware to the rescue
  - Atomic instructions: create a new CPU instruction that checks and sets a variable atomically
    - » Cannot be interrupted! (toss the problem to hardware designers)

```
struct lock {
    bool held = 0; // 0 === false
}
void acquire (lock) {
    while (test-and-set(&lock->held));
}
void release (lock) {
    lock->held = 0; // 0 === false
}
```

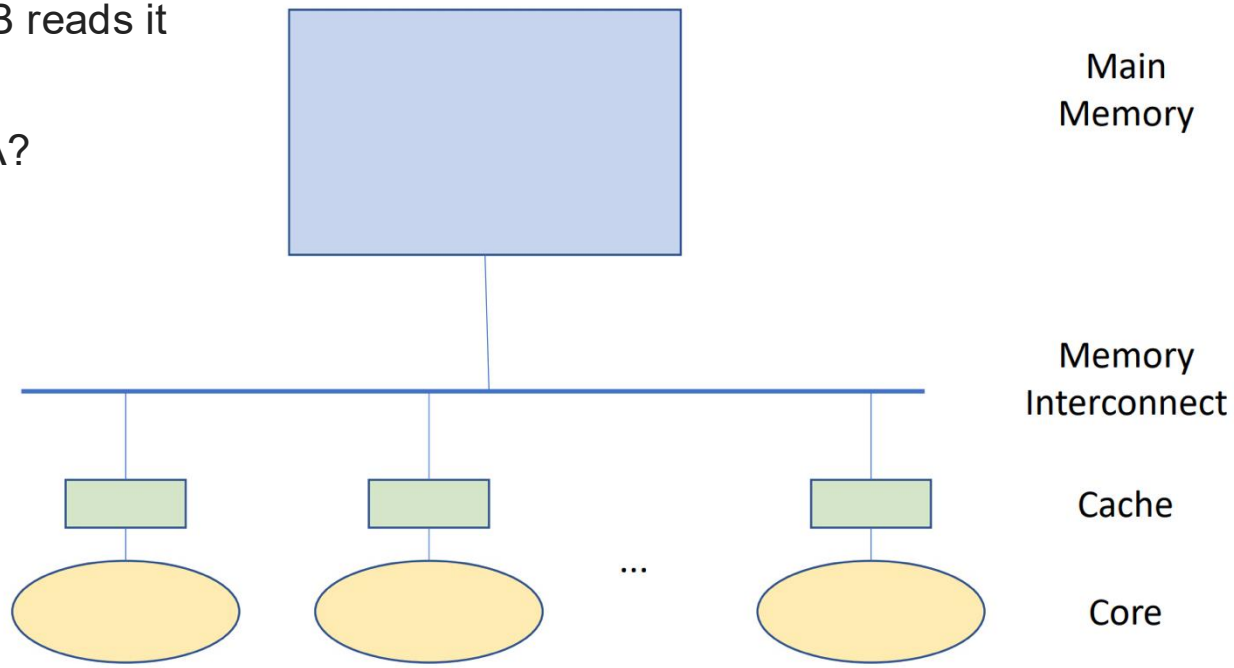
MCS Lock: cache effects on sync perf

- A program with lots of concurrent threads can still have poor performance on a multiprocessor:
  - Overhead of creating threads, if not needed
  - Lock contention: only one thread at a time can hold a given lock
  - Shared data protected by a lock may ping back and forth between cores
  - False sharing: communication between cores even for data that is not shared (but resides in same cache line)



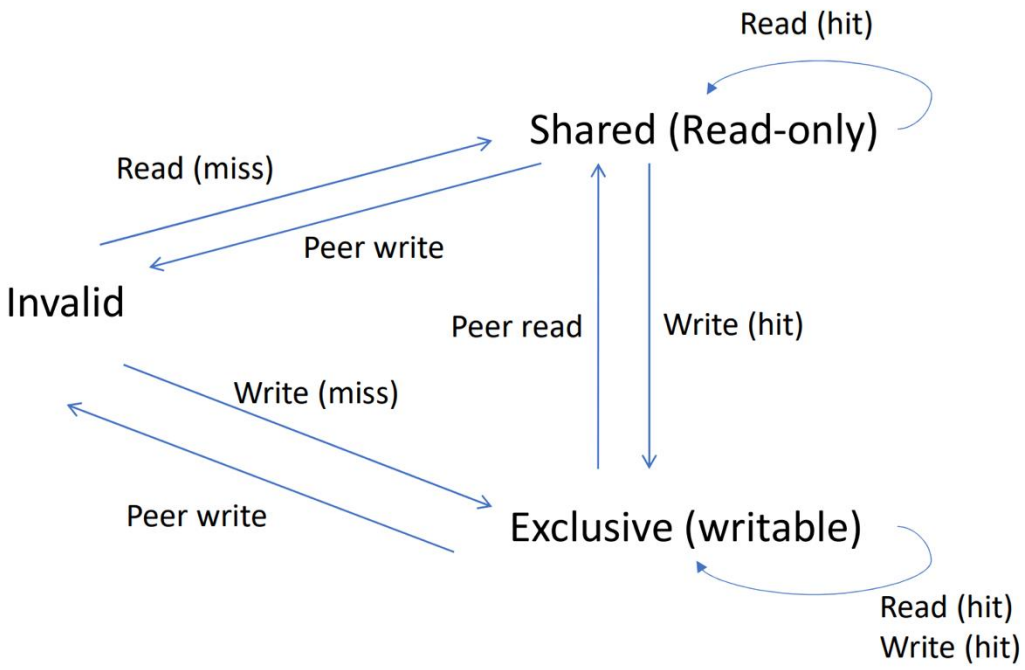
MCS Lock: cache coherent

- Scenario
  - Thread A modifies data inside a critical section and releases lock
  - Thread B acquires lock and reads data
- Easy if all accesses go to main memory
  - Thread A changes main memory; thread B reads it
- With (separate) caching
  - What if new data is cached at processor A?
  - What if old data is cached at processor B



MCS Lock: cache coherent

- Cache coherence: system behaves as if there is one copy of the data
  - If data is only being read, any number of caches can have a copy
  - If data is being modified, at most one cached copy
- On write: (get ownership)
  - Invalidate all cached copies, before doing wri
  - Modified data stays in cache ("write back")
- On read:
  - Fetch value from owner or from memory
- How?
  - Inter-processor communications



MCS Lock: cache coherent overhead

- Array(s) of 1K counters, each protected by a separate spinlock
  - Array small enough to fit in cache
- Test 1: one thread loops over array **51 cycles**
- Test 2: two threads loop over different arrays **52 cycles**
- Test 3: two threads loop over single array **192 cycles**
- Test 4: two threads loop over alternate elements in single array **127 cycles**

```
Class Counter {
    int lock;
    int value;
}
```

```
// A counter protected by a spinlock
Counter::Increment() {
    while (test_and_set(&lock))  ;      // spinlock acquire
    value++;                            // increment
    memory_barrier();                   // push updated values
    lock = FREE;                        // spinlock release
}
```

MCS Lock: cache coherent overhead

- The Problem with Test-and-Set

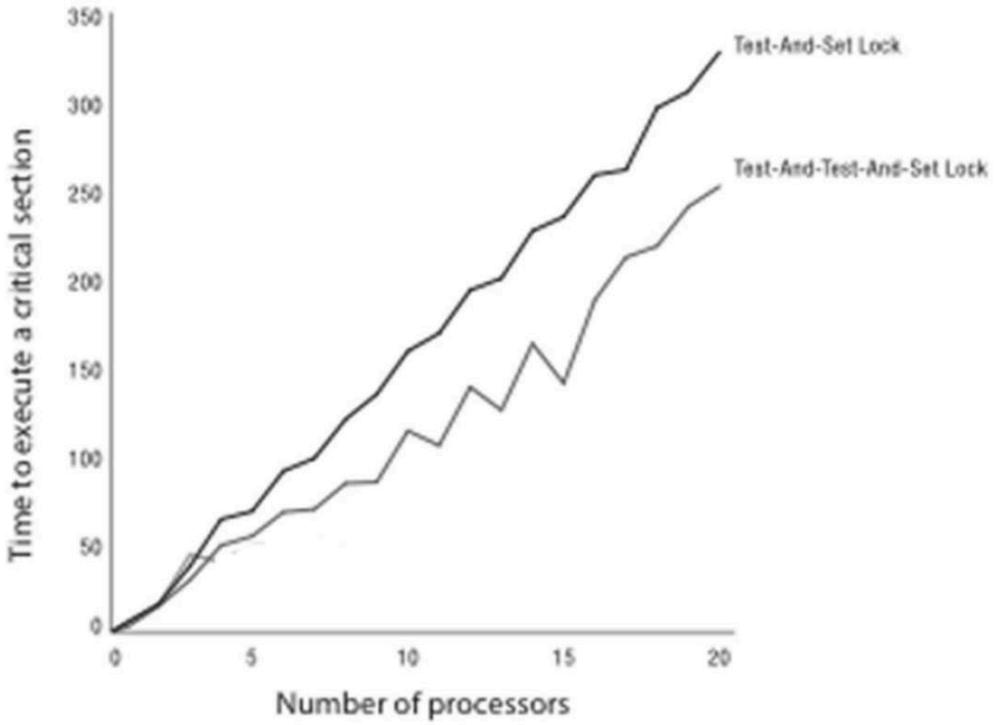
```
Counter::Increment() {
   while (test_and_set(&lock));        // this is a write!
   value++;
   memory_barrier();
  lock = FREE;                         // so is this…
}
```

- What happens if many processors try to acquire the lock at the same time?
  - Threads trying to get lock acquire cache line ownership
  - Hardware doesn't prioritize FREE

MCS Lock: test-and-test_and_set

- Try read first, if likely to succeed, then try write

```
struct lock {
    bool held = 0; // 0 === false
}
void acquire (lock) {
    while (lock->held ||
      test-and-set(&lock->held));
}
void release (lock) {
    lock->held = 0; // 0 === false
}
```



MCS Lock: exponential backoff

- The total amount of network traffic caused by busy-waiting on a test_and_set lock can be reduced further by introducing delay on each processor between consecutive probes of the lock.
  - Reduce competition, reduce contention, reduce traffic
  - How long? Random is the best, but exponential back-off gives some good “randomness”

```
struct lock {
    bool held = 0; // 0 === false
    unsigned delay = 1;
}
void acquire (lock) {
    while (lock->held ||
      test-and-set(&lock->held)) {
        pause(delay);
        delay = delay * 2;
    }
}
void release (lock) {
    lock->held = 0; // 0 === false
}
```

MCS Lock: ticket lock

- Test-and-test_and_set lock could still introduce short burst of contentions when unlocked, and there is not fairness guarantees
- Ticket lock
  - One write to acquire the ticket
  - Reads on the waiting/spinning

```
struct lock {
    atomic_unsigned now_serving;
    atomic_unsigned next;
}
void acquire (lock) {
    unsigned ticket = fetch_and_inc(&lock->next);
    while (ticket != now_serving);
}
void release (lock) {
    lock->now_serving++;
}
```

MCS Lock: ticket lock with pause

- Ticket lock could still have substantial memory and network contention through polling of a common location.
  - Reads on the waiting/spinning
- Pause still work, and even better here
  - We have a reasonable estimation!

```
struct lock {
    atomic_unsigned now_serving;
    atomic_unsigned next;
}
void acquire (lock) {
    unsigned ticket = fetch_and_inc(&lock->next);
    while (ticket != now_serving) {
        pause(ticket - now_serving);
    }
}
void release (lock) {
    lock->now_serving++;
}
```

MCS Lock: array-base queue lock

- Let's remove the shared variable (now_serving)

```
type lock = record
    slots : array [0..numprocs -1] of (has_lock, must_wait)
        := (has_lock, must_wait, must_wait, ..., must_wait)
        // each element of slots should lie in a different memory module
        // or cache line
    next_slot : integer := 0

// parameter my_place, below, points to a private variable
// in an enclosing scope

procedure acquire_lock (L : ^lock, my_place : ^integer)
    my_place^ := fetch_and_increment (&L->next_slot)
        // returns old value
    if my_place^ mod numprocs = 0
        atomic_add (&L->next_slot, -numprocs)
        // avoid problems with overflow; return value ignored
    my_place^ := my_place^ mod numprocs
    repeat while L->slots[my_place^] = must_wait      // spin
    L->slots[my_place^] := must_wait                  // init for next time

procedure release_lock (L : ^lock, my_place : ^integer)
    L->slots[(my_place^ + 1) mod numprocs] := has_lock
```

Fig 3. Anderson's array-based queuing lock.

MCS Lock: list-based queue lock

- Wait in a linked list
  - guarantees FIFO ordering of lock acquisitions;
  - spins on locally-accessible flag variables only;
  - requires a small constant amount of space per lock; and
  - works equally well (requiring only O(1) network transactions per lock acquisition) on machines with and without coherent caches

- Background: compare-and-swap

```
bool compare_and_swap (T *addr, T compare, T update) {
    if (*addr == compare) {
        *addr = update;
        return true;
    }
    return false;
}
```

MCS Lock: design

- MCS lock maintains a list of threads waiting for the lock
  - tail is reference to the last thread in list
  - The lock is free if tail is NULL
  - Otherwise, the lock is in use
    - » Thread at the head of the list holds the lock
- Threads spin on their private locked flags
- Lock is handed off by the thread releasing the lock:
  - next->locked = FALSE;

```
struct mcs_lock {
    struct qentry {
        // user must declare one of these to lock
        std::atomic<qentry*> next;
        std::atomic<bool> locked;
    };

    std::atomic<qentry*> lk_;   // points at tail

}
```

MCS Lock: design

- Lock:
  - Use compare-and-swap to insert into tail

```
struct mcs_lock {
    struct qentry {
        // user must declare one of these to lock
        std::atomic<qentry*> next;
        std::atomic<bool> locked;
    };

    std::atomic<qentry*> lk_;   // points at tail

    void lock(qentry& w) {
        w.next = nullptr;
        qentry* prev_tail = lk_.exchange(&w);  // mark self as tail
        if (prev_tail) {                      // previous tail exists
            w.locked = true;
            prev_tail->next = &w;            // link to previous tail
            while (w.locked) {
                pause();
            }
        }
    }
}
```

MCS Lock: design

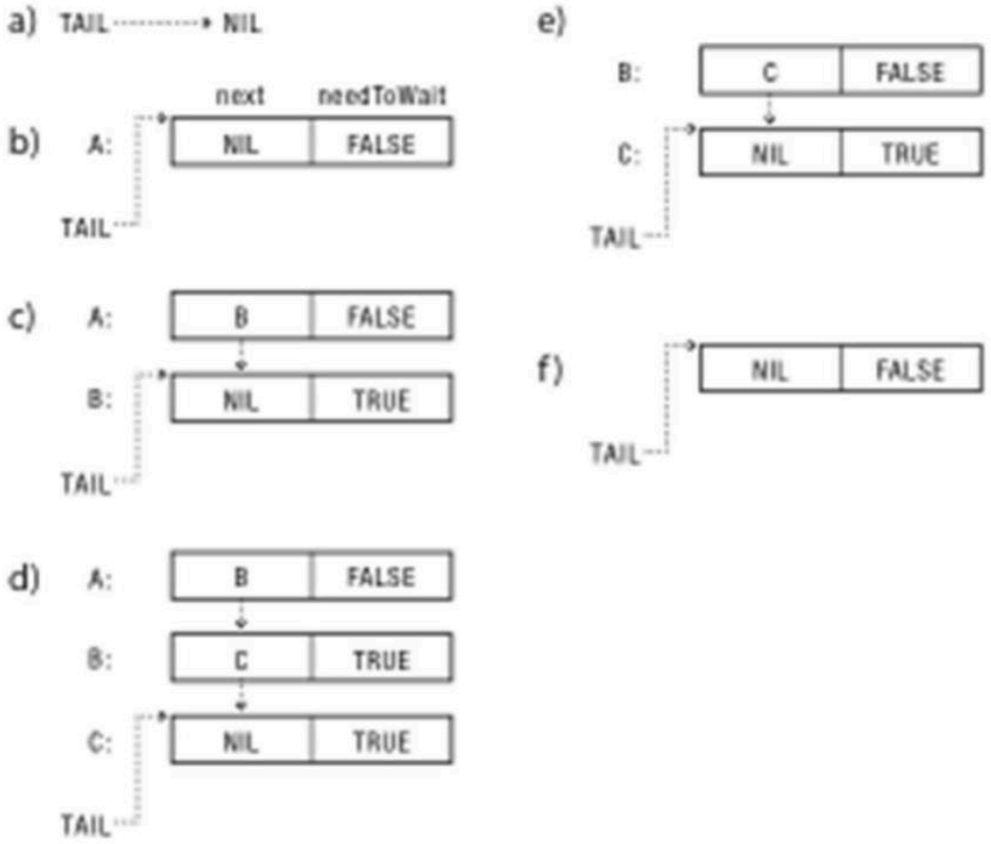
- Unlock:
  - Am I the tail?
  - Wait if linking in action

```
struct mcs_lock {
    struct qentry {
        // user must declare one of these to lock
        std::atomic<qentry*> next;
        std::atomic<bool> locked;
    };

    std::atomic<qentry*> lk_;   // points at tail

    void unlock(qentry& w) {
        if (!w.next) {                    // maybe become unlocked
            if (lk_.compare_exchange_weak(&w, nullptr)) {
                return;
            }
            while (!w.next) {   // wait for next tail to link self
                pause();
            }
        }
        w.next->locked = false;
    }
}
```

MCS Lock: in action



MCS Lock: performance

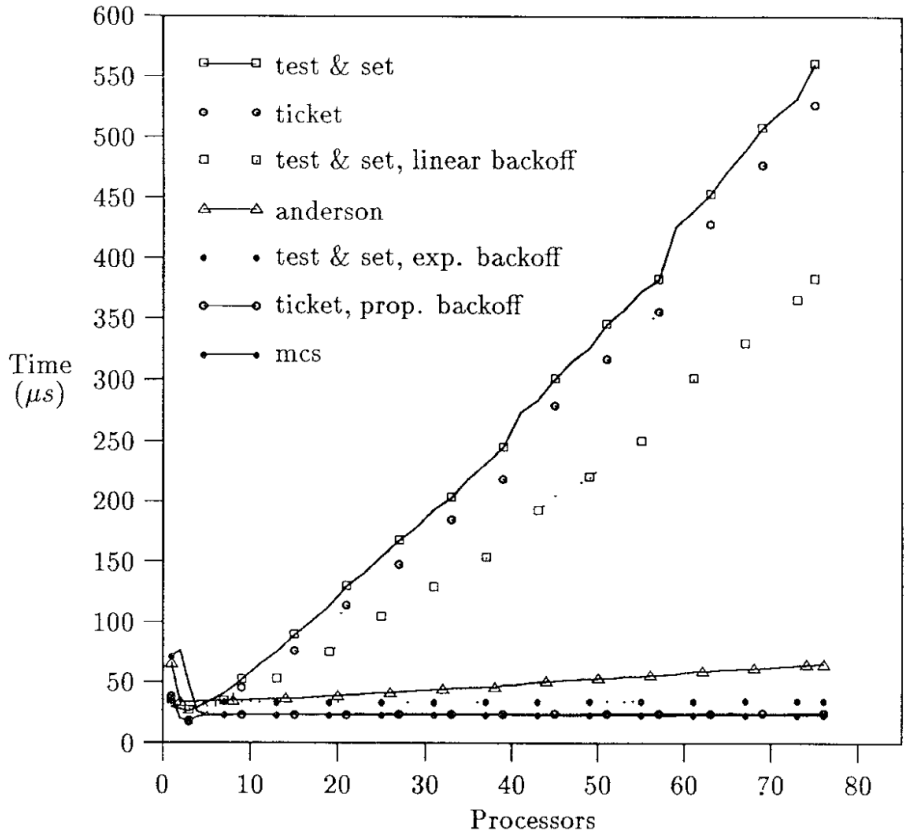
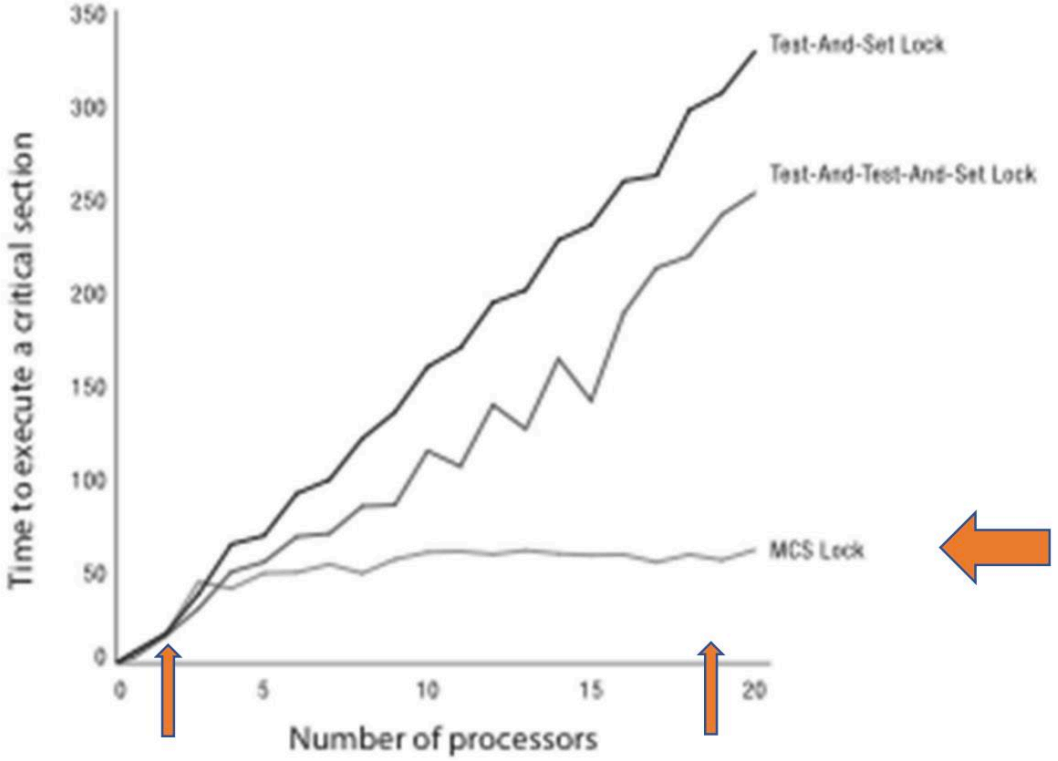


Fig. 15. Performance of spin locks on the Butterfly (empty critical section)



Read-Copy-Update

- Three key requirements of the kernel:
  1. Useful forward progress for concurrent readers, even during updates;
     - » File access is common, so ideally, threads reading from the cache would not interfere with each other or be impeded by threads performing updates.
  2. Low computation and storage overhead;
     - » The kernel can cache more than 4 million dentry objects and and 1 million ext4 inodes, an overhead of more than a few bytes per object is unacceptable.
     - » For performance-critical data structures like SELinux AVC that the kernel might access several times during a system call, incurring a single cache miss (WHY?), which can cost hundreds of cycles, would double the cost of accessing.
  3. Deterministic completion time.
     - » Deterministic completion time is is critical to real-time response, and benefits non-maskable interrupt (NMI) handlers, where spinlock may lead to deadlock (WHY?) and OCC can lead to many retries.

RCU vs. Other Synchronization

- Read-write locks, Linux local-global locks, transactional memory
  - None of these primitives provide useful concurrent forward progress to both read and write operations acting on the same data.
  - They all impose a storage overhead.
  - Read-write locks and local-global locks use **expensive** atomic instructions or memory barriers during acquisition and release.
- In contrast, in the most aggressive RCU implementations (for example, those used by Linux-kernel servers), RCU readers might execute exactly the same sequence of machine instructions that would be executed by a single-threaded traversal of that same data structure.
  - How?

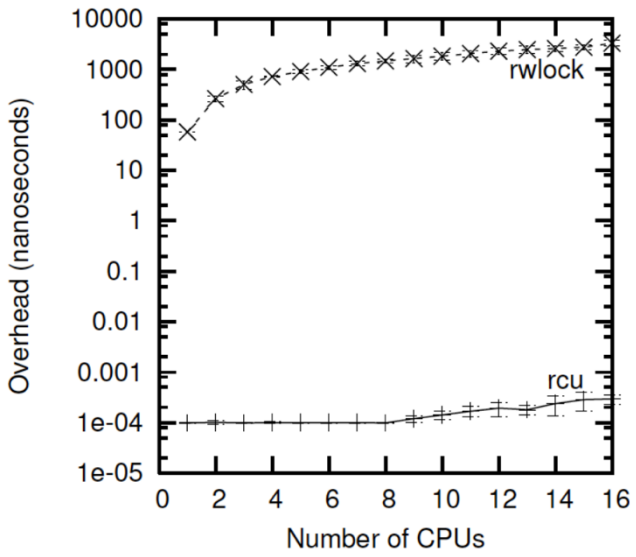


Figure 8: The overhead of entering and completing an RCU critical section, and acquiring and releasing a read-write lock.

RCU: why interesting?

- Example of a synchronization primitive that is:
  - Lock free (mostly/for reads)
  - Tuned to a common access pattern
  - Making the common case fast
- What is this common pattern?
  - A lot of reads
  - Writes are rare
    - » Prioritize writes
  - Stale copies are short lived – time heals all wounds
  - Ok to read a slightly stale copy
    - » But that can be fixed too

RCU: the typical work

- Reader: traversing a data-structure connected by pointers (e.g., a link list) **in a single direction**
- Writer: inserting / deleting a new entry
- Without synchronization:

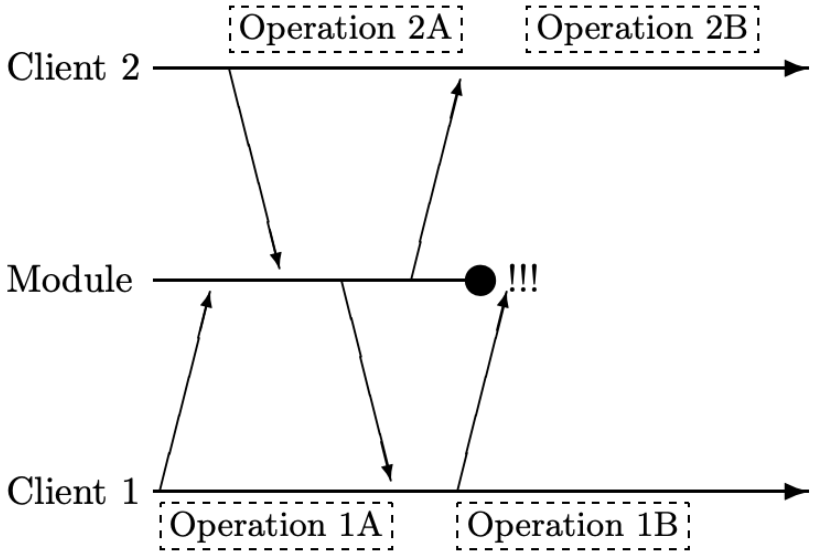


Figure 1: Race Between Teardown and Use of Service

RCU: the high-level idea

- Replace pointers to a data structure with pointers to a new version
  - For write: Copy-then-Update
  - For deletion: new version == nullptr
  - Is this replacement atomic?
  - Why is this okay?
- Wait for all previous readers to complete their RCU read-side critical sections.
- At this point, there cannot be any readers who hold reference to the old data structure, so it now may safely be reclaimed.

RCU: the typical work with RCU

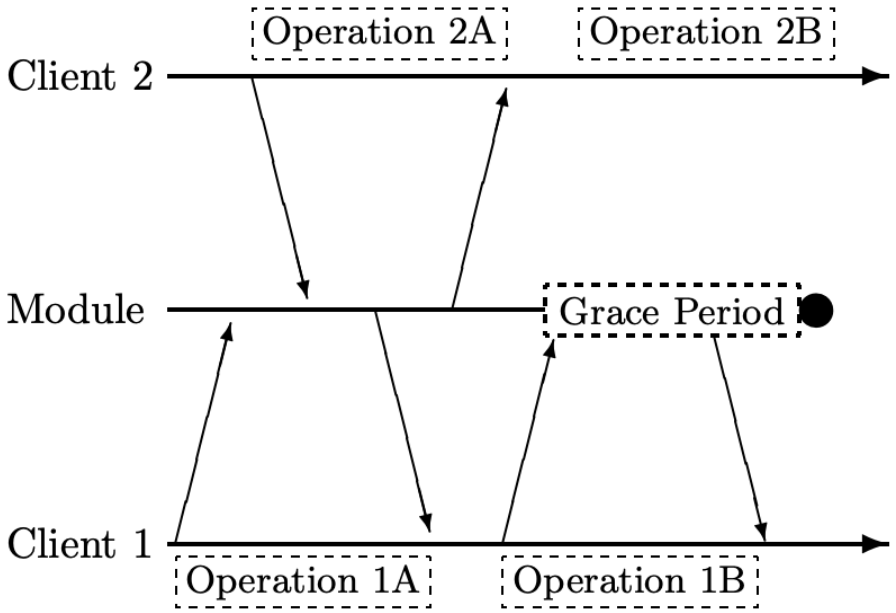


Figure 2: Read-Copy Update Handling Race

- Grace period before 1B finishes with the module
- 2B is told the module is gone – no problem

RCU: double linked list deletion

- Why spin_lock?
- Do we need spin_lock for single link?

```
void delete(struct el *p)
{
    spin_lock(&list_lock);
    p->next->prev = p->prev;
    p->prev->next = p->next;
    spin_unlock(&list_lock);
    call_rcu(&p->my_rcu_head,
    my_free, p);
}
```

Phase 1

Schedule phase 2 after quiescence

Figure 4: Read-Copy Dequeue From Doubly-Linked List

RCU: double linked list deletion

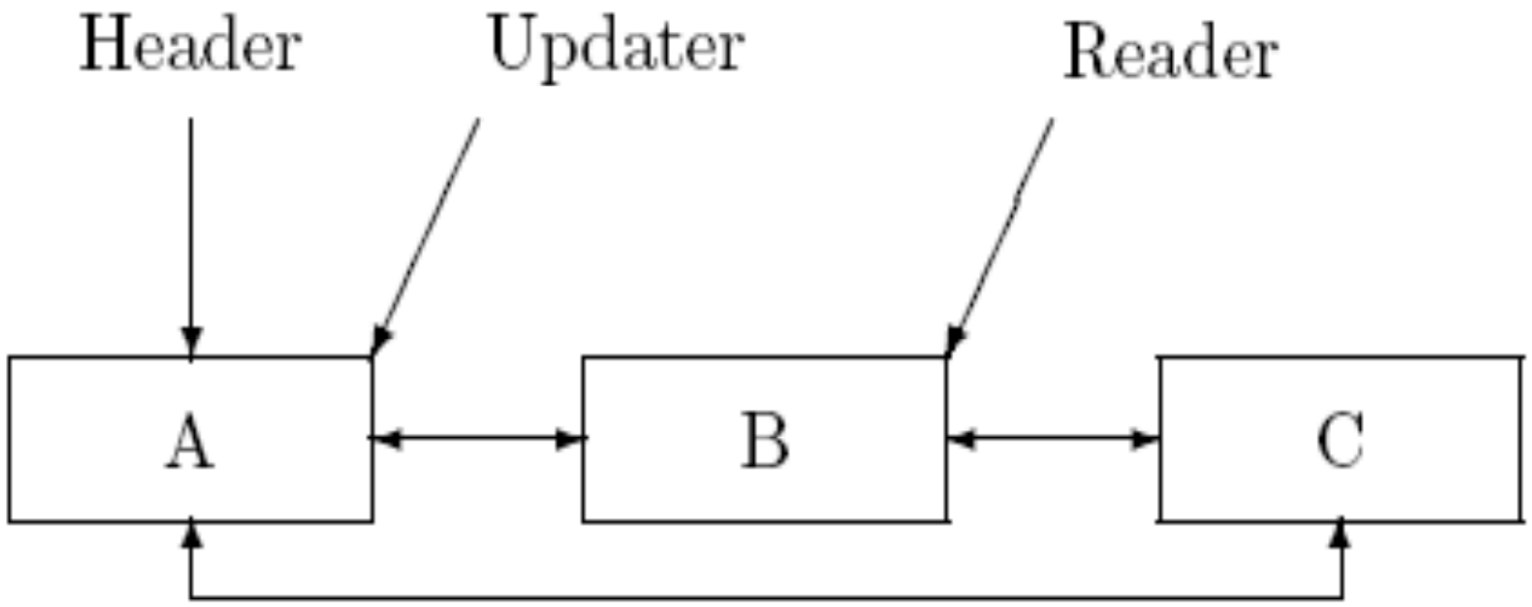


Figure 11: List Initial State

RCU: double linked list deletion

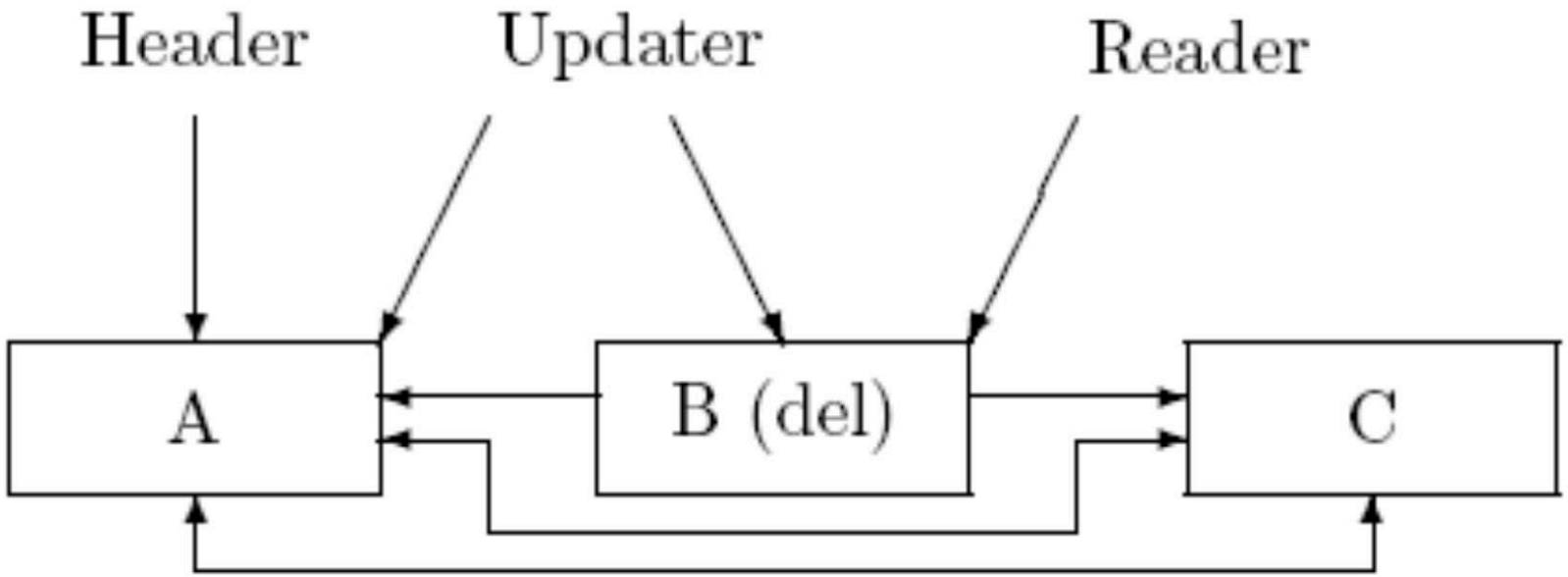


Figure 12: Element B Unlinked From List

RCU: double linked list deletion

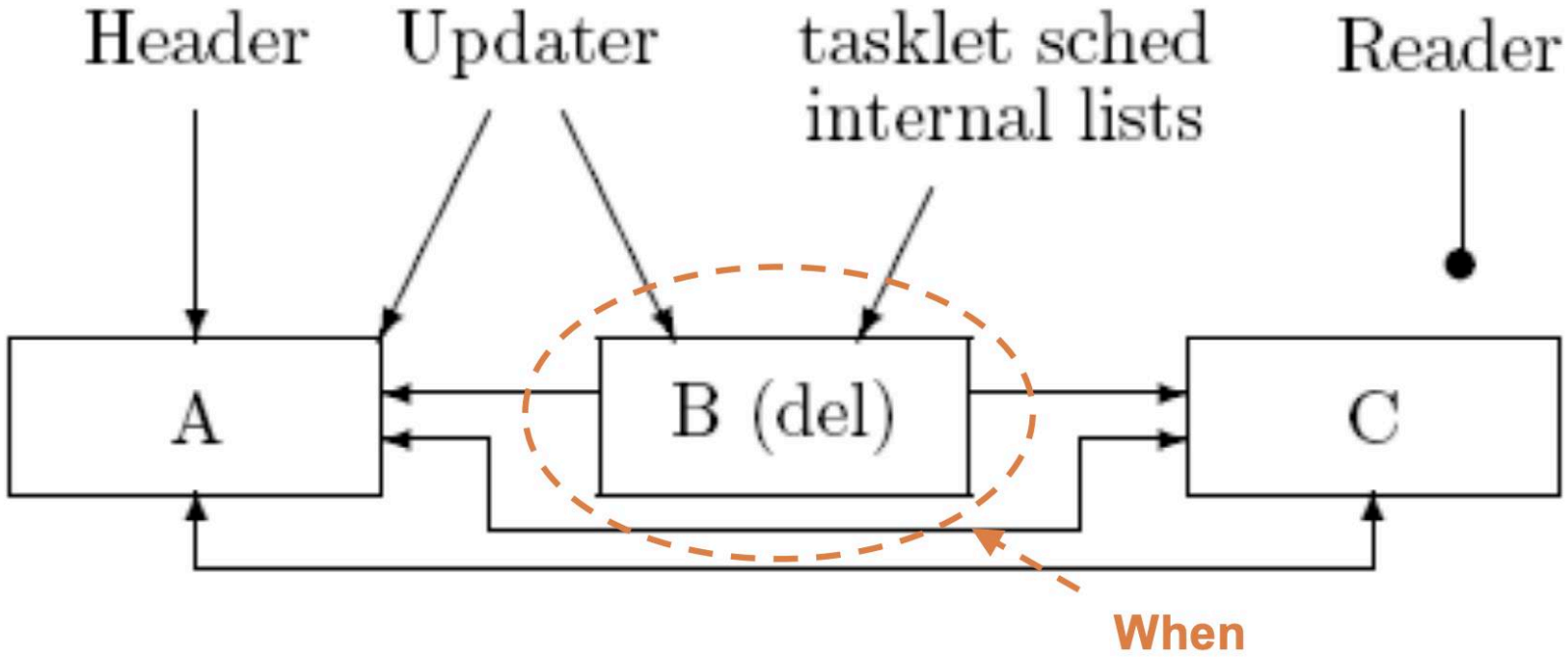


Figure 13: List After Grace Period

RCU: double linked list deletion

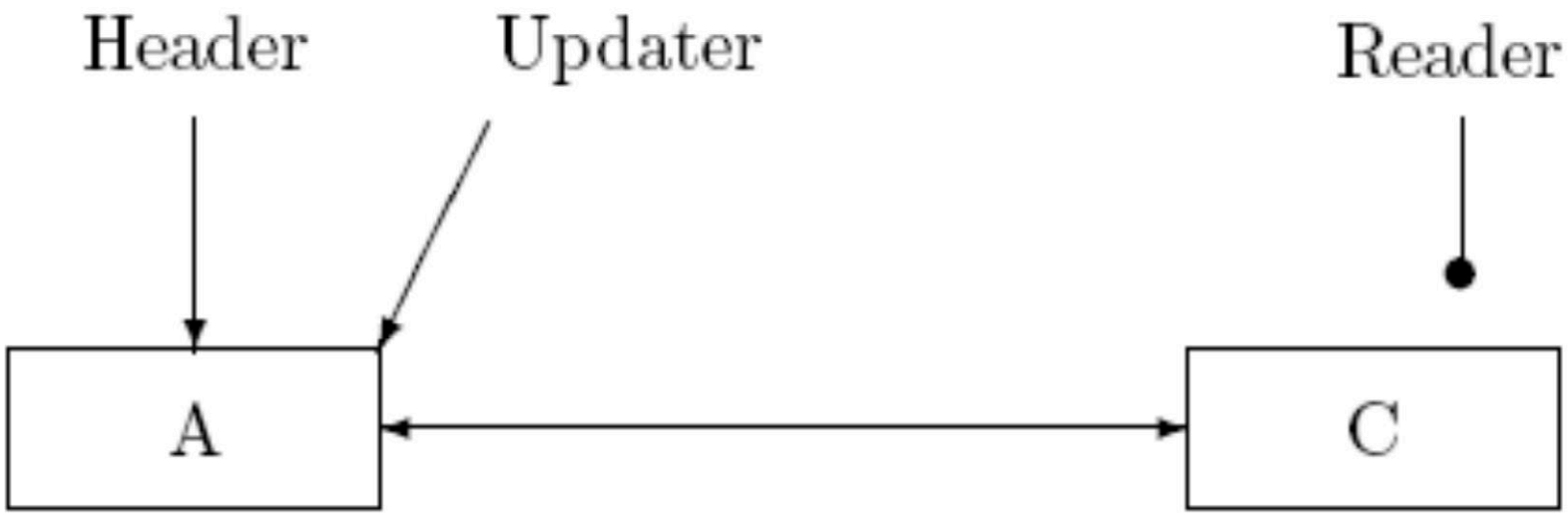
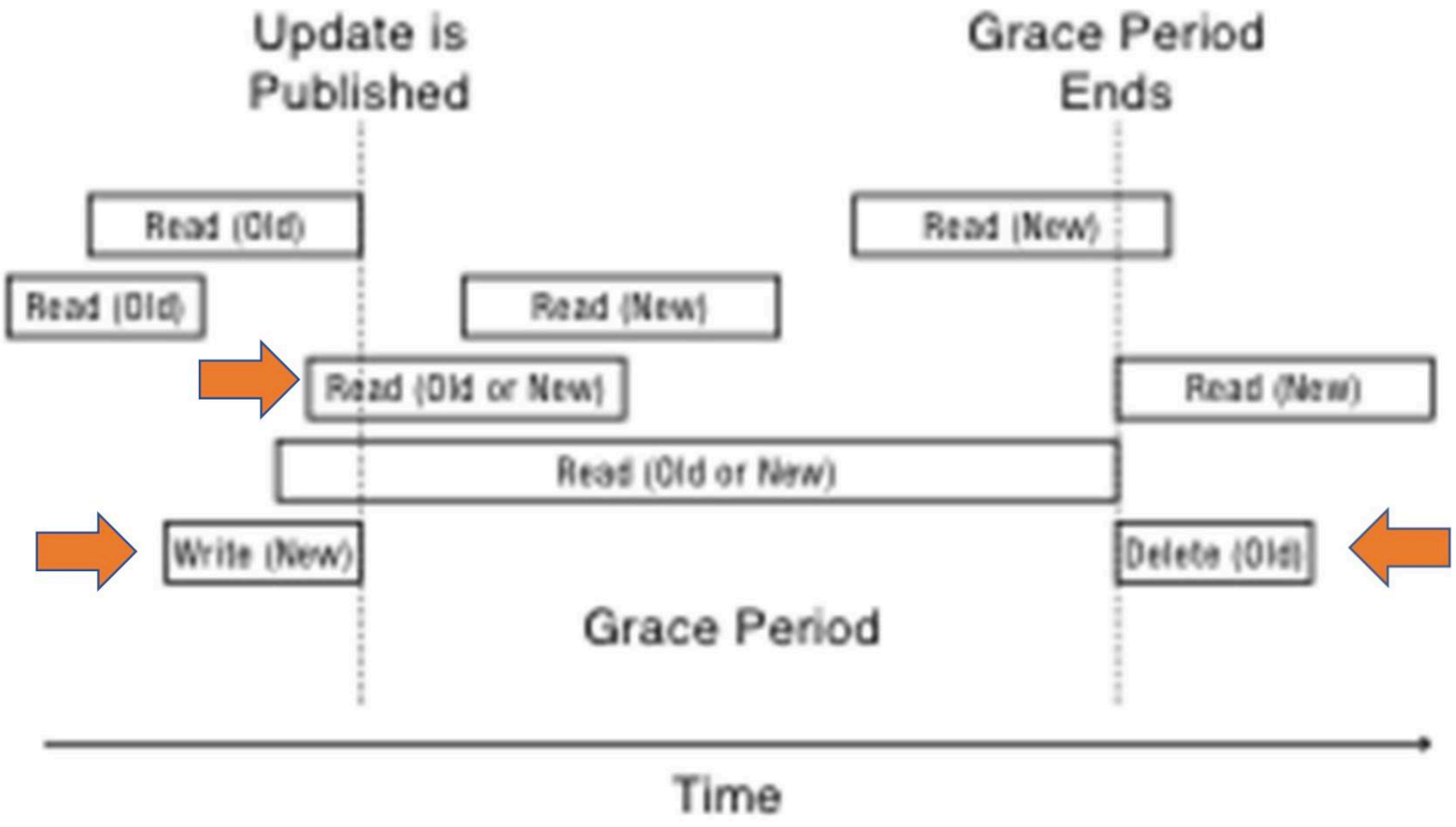


Figure 14: List After Element B Returned to Freelist

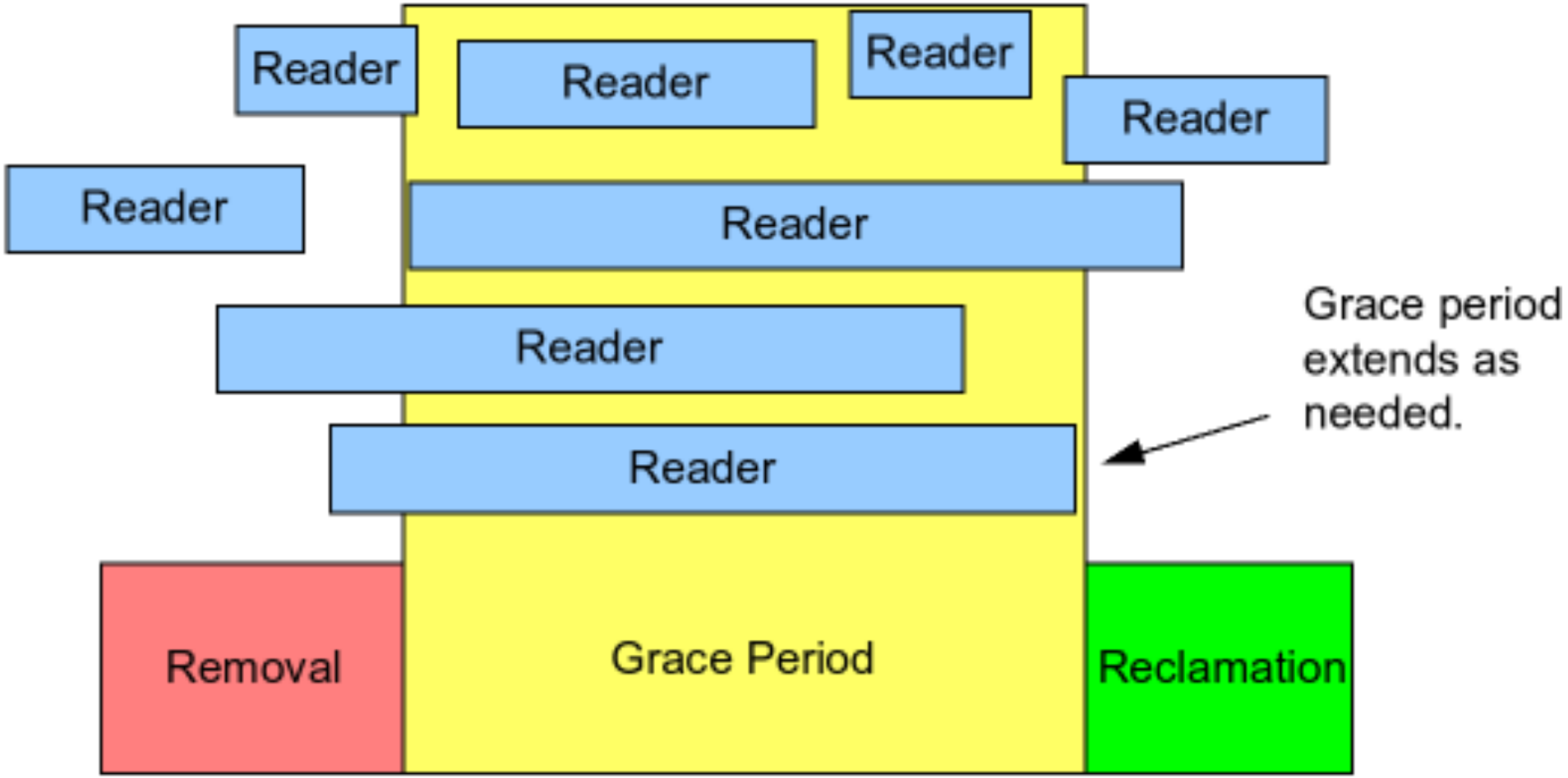
RCU: the jargon

- **Quiescent State**
  - A point in the code where all of the current entity's temporary variables that were in use before a specified time are dead. No more reference (pointer) to the old data.
    - » In a non-preemptive Linux kernel, a context switch is a quiescent state for CPUs.
    - » In a preemptive Linux kernel, a voluntary context switch is a quiescent state, but for threads.
- **Grace Period**
  - Time interval during which all entities (CPUs or tasks, as appropriate) pass through at least one quiescent state.
  - Note that any time interval containing a grace period is itself a grace period.

RCU: grace period



RCU: grace period



RCU: how to detect quiescence?

- Idea of grace periods
  - Readers of old information will eventually leave
  - Exploit context switches
    - » Threads do not hold OS locks / temp references across context switches
- How do we identify?
  - Paper goes into many alternatives and evaluates them (polling; counters; …)
  - Batching to reduce cost
  - Force context switch?
    - » Expensive and some tasks are not preemptible
  - IPI for time sensitive tasks

RCU: APIs

`rcu_read_lock()`	Begin an RCU critical section.
`rcu_read_unlock()`	Complete an RCU critical section.
`synchronize_rcu()`	Wait for existing RCU critical sections to complete.
`call_rcu(callback, arguments...)`	Call the callback when existing RCU critical sections complete.
`rcu_dereference(pointer)`	Signal the intent to deference a pointer in an RCU critical section.
`rcu_assign_pointer(pointer_addr, pointer)`	Assign a value to a pointer that is read in RCU critical sections.

Async, no waiting

Figure 3: Summary of the Linux RCU API.

RCU: implementations

```
void rcu_read_lock()
{
  preempt_disable();
}

void rcu_read_unlock()
{
  preempt_enable();
}

void synchronize_rcu(void)
{
  int cpu;

  for_each_online_cpu(int cpu)
    run_on(cpu);
}
```

NOP for server kernel (non-preemptive)

NOP for server kernel (non-preemptive)

In practice, Linux implements synchronize_rcu by waiting for all CPUs in the system to pass through a context switch, instead of scheduling a thread on each CPU.

Figure 2: A simplified version of the Linux RCU implementation.

RCU: implementations

- RCU helps developers manage reordering with `rcu_dereference` and `rcu_assign_pointer`
  - Readers use `rcu_dereference` to signal their intent to read a pointer in a RCU critical section
  - Updaters use `rcu_assign_pointer` to mutate these pointers
  - These two primitives contain architecture-specific memory-barrier instructions and compiler directives to enforce correct ordering
  - Both primitives reduce to simple assignment statements on sequentially consistent systems
- Re-ordering?

```
struct A {
  int data;
  struct A *next;
};
```

```
struct A *old = list_head;
struct A *new = kmalloc();
// copy
new->data = old->data;
new->next = old->next;
// update
list_head = new;
```

```
struct A *old = list_head;
struct A *new = kmalloc();
// copy
new->data = old->data;
new->next = old->next;
// update
rcu_assign_pointer(&list_head, new);
// =>
// barrier();
// list_head = new;
```

RCU: vs refcount

- Get a temporary copy of data

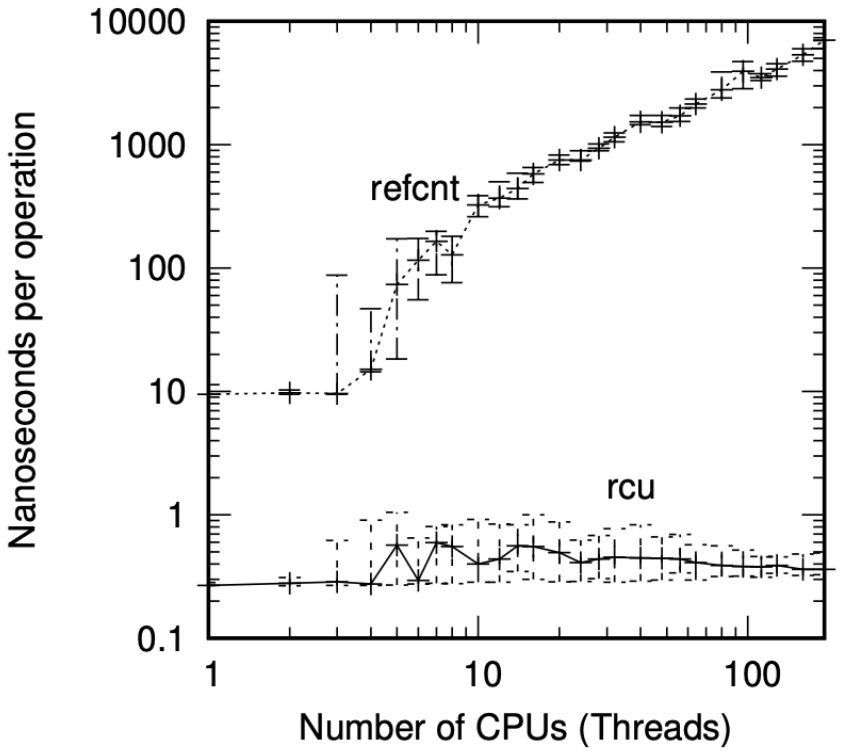


Figure 5: The overhead of RCU readers compared to that of a shared-integer reference count.

RCU: the paper

- Really challenging paper to read
  - Written by OS hackers (good thing!)
  - Mixes fundamentals and implementations
    - » We have to try to step back and identify them
  - Too many ideas/alternatives
    - » Better just to focus on one or two?
  - Back end of the paper is a survey
- The ten- and eighteen-years review papers are easier to read

Synchronization

- Takeaways at graduate level
  - What is memory reordering and barrier?
  - Atomic is not free!
  - Cache coherent is expensive
  - Lock is not the only answer
- Best practice
  - Data race avoidance → true parallel algorithms
  - Lock avoidance → if contention is low, try OCC
  - Reducing contention → lock and cache